KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU, DTE Code: EN6634)
(Accredited by NAAC with 'A+' Grade)

DEPARTMENT OF COMPUTER ENGINEERING

Lab Manual

Web Development Laboratory

VSE-281-COM

AY-2025-2026

SE-A- SEM-II

Prepared By - **Prof. U.N.Yadav**

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
Practical Plan A.Y.2025-26 Semester-II

Department of Computer Engineering				
Class: SE DIV-A		**Subject: Web Development Laboratory**		
Name of Staff : Prof. U.N.Yadav		**Code:**	**VSE-281-COM**	
Examination Scheme: Term Work : 25 Marks Practical : 25 Marks				
Experiment No.	**Title of Assignment/Experiment**	**Batch**	**Planned Date**	**Conducted Date**
Web Development Laboratory				
1	Create a simple HTML page displaying personal details using text formatting tags.	S1		
		S2		
2	Design a web page that includes an image, hyperlink, and a nested table.	S1		
		S2		
3	Create a form with fields: name, email, gender, date of birth, and submit button.	S1		
		S2		
4	Apply internal, external, and inline CSS to style a web page with headings and tables.	S1		
		S2		
5	Develop a responsive web page using Bootstrap Grid and Components	S1		
		S2		
6	Write a JavaScript program to validate form inputs (e.g., email, empty fields).	S1		
		S2		

7	Create a web page that uses JavaScript to display dynamic content using DOM.	S1 S2		
8	Write a JavaScript program for a simple calculator using functions and switch-case.	S1 S2		
9	Design a PHP script to display "Welcome" message and current date & time.	S1 S2		
10	Write a PHP program to accept form input and display it using the POST method.	S1 S2		
11	Implement a PHP program for string manipulation (e.g., reverse, length, substring).	S1 S2		
12	Create a PHP script to store and display values in an array.	S1 S2		

Subject Incharge **HOD**

Instructions for writing journals

1) Attach front page containing Experiment No. , Title, and Performed date for every experiment. Specimen copy of the format for front page is given.

2) Write theory in detail from the points provided in the lab manuals.

3) Diagrams must be drawn on blank page with pencil only, do not use pen to draw diagram.

4) Use scales wherever necessary do not use free hand drawing.

5) Do not copy theory from manuals as it is in brief and given only for reference. Write your own theory.

6) Always attach conclusion at last.

Group A-Experiment No: 01	
Title: Create a simple HTML page displaying personal details using text formatting tags	
Date of performance:	**Date of Submission:**
Class: SE -A	**Roll No.:**
Signature:	

Experiment No. 1

Title:

Create a simple HTML page displaying personal details using text formatting tags.

Objective:

- To understand basic HTML structure and syntax.
- To learn and use text formatting tags in HTML.
- To display personal details in a structured, readable format.

Problem Statement

Design an HTML page that displays personal details such as name, address, contact information, academic details, hobbies, etc., using appropriate HTML text formatting tags.

Theory:

HTML (HyperText Markup Language) is used to create webpages.

Text formatting tags help make the text bold, italic, underlined, highlighted, superscript, subscript, etc.

Common Text Formatting Tags Used

Tag	Description
<b>	Makes text bold
<strong>	Indicates important text (bold + semantic importance)
<i>	Italics
<em>	Emphasized text

<u> Underlines text

<mark> Highlights text

<small> Makes text smaller

<sup> Superscript (e.g., 10th)

<sub> Subscript (e.g., H₂O)

<p> Paragraph

 Line break

<h1>.. <h6> Heading tags

<hr> Horizontal line divider

These tags help present information clearly and attractively.

Test Cases

Test Scenario	Expected Output
Display Name	Name appears in bold or heading
Display Address	Shown as paragraph with proper formatting
Add email/phone	Clearly visible and separated using ` `
Apply text formatting	Italic, bold, underline appear correctly
Page loads in browser	All formatted text visible without errors

Conclusion

The experiment successfully demonstrates how HTML text formatting tags can be used to present personal details in a structured and visually enhanced manner on a webpage.

Questions

1. What are text formatting tags in HTML?
2. Difference between `<b>` and `<strong>`?
3. What is the use of `<em>` tag?
4. How is `<br>` different from `<p>`?
5. What are semantic tags? Give examples.
6. Why are heading tags `<h1>`–`<h6>` important?

Experiment No: 02	
Title: Design a web page that includes an image, hyperlink, and a nested table	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No. 2

Title:

To design an HTML webpage that displays an image, a hyperlink, and a nested table for organizing structured data.

Objective:

- To understand how to embed images in a webpage.
- To learn how to create hyperlinks for navigation.
- To design tables and nested tables for displaying tabular data.
- To apply basic HTML tags for webpage layout.

Problem Statement

Create an HTML webpage that displays:

1. One image (any personal or sample image)
2. A hyperlink to another website or page
3. A table containing a nested table inside one of its cells

Ensure the layout is neat and well-organized

Theory:

A webpage often contains images, links, and structured data presented using tables. HTML provides simple tags to implement these elements.

Tags Used & Description

1. Image Tag

- **`<img>`**
  Used to display images on a webpage.
  Attributes:
  - `src` – image file path

  - `alt` – alternative text
  - `width`, `height` – size control

2. Hyperlink Tag

- **`<a>`**
  Used to link to another webpage, website, or location.
  Attributes:
  - `href` – target URL
  - `target="_blank"` – opens link in a new tab

3. Table Tags

HTML tables are created using rows and cells.

- `<table>` – defines a table
- `<tr>` – creates a row
- `<th>` – table header
- `<td>` – table data

Nested Table

A nested table is a table placed **inside a `<td>` cell** of another table.

Example concept:

```
<table>
  <tr>
     <td>
        <table> ... </table>
     </td>
  </tr>
</table>
```

Used to show complex or multi-level data.

These tags help present information clearly and attractively.

Test Cases

Test Scenario	**Expected Output**
Image path is correct	Image loads properly
Image path incorrect	Browser shows broken image icon with alt text
Click hyperlink	Redirects to specified URL
Nested table added inside parent table	Nested table appears correctly inside a cell
Table borders applied	Both tables visible clearly

Conclusion

Image embedding, hyperlinking, and table creation were successfully implemented.
Nested tables help organize hierarchical or multi-level data effectively in a webpage layout.

Questions

1. What is the purpose of the `<img>` tag?
2. What is the use of the `alt` attribute in an image?
3. How do you create a hyperlink in HTML?
4. What is a nested table?
5. Difference between `<th>` and `<td>`?
6. Name any two attributes of the `<img>` tag.
7. Why are hyperlinks important in web development?

Experiment No: 03	
Title: Create a form with fields: name, email, gender, date of birth, and submit button.	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 3

Title:

Create a form with fields: name, email, gender, date of birth, and submit button.

Aim

To design an HTML form that collects user information such as name, email, gender, date of birth, and allows submission using a submit button.

Objectives

- To understand HTML `<form>` tag and various form input elements.
- To learn how to use text fields, radio buttons, date pickers, and submit buttons.
- To understand form structure and user input collection.

Problem Statement

Create a web page containing a simple HTML form with fields for Name, Email, Gender (Male/Female), Date of Birth, and a Submit button. The form should be properly structured using form tags and appropriate input types.

Theory

HTML Form

The `<form>` tag is used to collect user input. Form elements allow users to enter data, which can later be processed by a server.

Tags Used

Tag	Description
`<form>`	Defines a form area where user input is taken.
`<label>`	Displays a label for an input field.
`<input>`	Used for text, email, radio, date, and submit controls.
`type="text"`	Used to accept simple text input.
`type="email"`	Ensures email format input.
`type="radio"`	Allows selection of one option from many.
`type="date"`	Used for date picker.
`type="submit"`	Creates a button to submit form.
` `	Line break for layout spacing.

Test Cases

Test Case No.	Input	Expected Output
1	Name: “Riya”, Email: “riya@gmail.com”, Gender: Male, DOB: 12-05-2004	Form accepts all valid inputs and proceeds on submit
2	Name empty	Displays error (if validation applied) or form submits empty value
3	Incorrect email format: “abc.com”	Browser highlights invalid email field
4	Gender not selected	Form may submit but gender remains unselected
5	Only name and email filled	Form still submits (if no validation applied)

Conclusion

The experiment successfully demonstrates how to create a basic HTML form using various input controls such as text fields, radio buttons, date pickers, and submit buttons. It helps in understanding user input collection on web pages.

Questions

1. What is the purpose of the `<form>` tag in HTML?
2. Why is `type="email"` preferred over a normal text field for email?
3. What is the difference between radio buttons and checkboxes?
4. Why do we use `<label>` tags in a form?
5. What type of input is used for selecting a date?

Experiment No: 04	
Title: Apply internal, external, and inline CSS to style a web page with headings and tables.	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 4

Title:

Apply internal, external, and inline CSS to style a web page with headings and tables.

Aim

To apply inline, internal, and external CSS styles to an HTML web page containing headings and tables.

Objectives

- To understand various types of CSS: inline, internal, and external.
- To learn how CSS enhances the appearance of web pages.
- To apply style properties to headings and tables.
- To differentiate between CSS types through implementation.

Problem Statement

Design a webpage with headings and a table. Apply inline CSS to one element, internal CSS to other elements, and external CSS for the remaining elements. Demonstrate the styling effect of each CSS type.

Theory

CSS (Cascading Style Sheets)

CSS defines how HTML elements are displayed on the screen. It controls layout, colors, fonts, spacing, borders, and overall presentation of webpages.

Types of CSS

1. Inline CSS

- Written directly inside an HTML tag using the **style** attribute.
- Example:
```
<h1 style="color:red;">Heading</h1>
```

2. Internal CSS

- Written inside `<style>` tag in the `<head>` section of HTML.
- Used for styling a single page.
- Example:
- `<style>`
- `    h2 { color: blue; }`
- `</style>`

3. External CSS

- Written in a separate `.css` file and linked using the `<link>` tag.
- Used for styling multiple pages consistently.
- Example:
```
<link rel="stylesheet" href="style.css">
```

Tags Used

Tag	**Description**
`<html>`	Structure of webpage
`<head>`	Contains metadata and internal CSS
`<style>`	Internal CSS block
`<link>`	Links external CSS file
`<body>`	Contains main content
`<h1>, <h2>, <h3>`	Heading tags
`<table>`	Creates a table
`<tr>`	Table row

Tag	Description
`<th>`	Table header cell
`<td>`	Table data cell
`style=""`	Inline CSS attribute

Test Cases

Test Case No.	Input	Expected Output
1	Apply inline CSS to `<h1>`	Heading appears with inline styling (e.g., red color, large font)
2	Apply internal CSS to a table	Table formatting such as borders, padding applied
3	Apply external CSS to `<h2>`	Heading appears with external CSS file styling
4	Remove external CSS link	Page loads but without external styling
5	Modify internal style	Changes immediately visible on the page

---

Conclusion

The experiment demonstrates the use of inline, internal, and external CSS to style webpage content. Each CSS type serves a specific purpose: inline for quick local changes, internal for single-page styling, and external for website-wide consistency. Understanding these enhances webpage presentation and maintainability.

Questions

1. What is the difference between inline, internal, and external CSS?
2. Why is external CSS preferred when designing large websites?
3. Can inline CSS override internal and external CSS? Explain.
4. What is the purpose of the `<style>` tag in HTML?
5. How do you link an external CSS file to an HTML page?

Experiment No: 05	
Title: Develop a responsive web page using Bootstrap Grid and Components.	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 5

Title: Develop a responsive web page using Bootstrap Grid and Components.

Aim

To design and develop a responsive webpage using Bootstrap Grid System and Bootstrap Components.

Objectives

- To understand the Bootstrap Grid System for responsive layout design.
- To learn how to apply Bootstrap classes for styling.
- To use Bootstrap components such as Navbar, Buttons, Cards, etc.
- To create a mobile-friendly and responsive webpage.

Problem Statement

Create a responsive webpage using the Bootstrap framework. The webpage should include a grid layout, navigation bar, buttons, and at least one Bootstrap component such as a card or alert. The webpage must properly adjust its layout across different screen sizes (mobile, tablet, desktop).

Theory

Bootstrap

Bootstrap is a popular CSS framework used to build responsive and mobile-first websites quickly and efficiently.

Bootstrap Grid System

- Based on **12-column layout**.
- Responsive breakpoints:
    - `col-` (extra small)

- `col-sm-` (small)
- `col-md-` (medium)
- `col-lg-` (large)
- `col-xl-` (extra large)

Example:
`<div class="col-md-6">` occupies 6 columns on medium screens.

Common Bootstrap Components

Component	Purpose
Navbar	Creates responsive navigation menu
Button (`btn btn-primary`)	Styled clickable button
Card	Box container with image, title, and text
Alert	Displays highlighted messages
Container / Row / Col	Used for page layout

Tags & Classes Used

Tag / Class	Description
`<link>`	Loads Bootstrap CSS
`<nav class="navbar">`	Navigation bar
`<div class="container">`	Main content wrapper
`<div class="row">`	Grid row
`<div class="col-md-4">`	Grid columns
`btn btn-primary`	Bootstrap button
`card`	Bootstrap card component

Tag / Class	Description
`alert alert-info`	Information alert
`img-fluid`	Responsive image class

Test Cases

Test Case No.	Input	Expected Output
1	View page on mobile	Columns stack vertically; navbar collapses
2	View page on laptop/desktop	Grid shows multiple columns side-by-side
3	Use Bootstrap class `btn btn-success`	Button appears styled in green
4	Remove Bootstrap link	Page becomes unstyled and loses layout
5	Resize browser window	Content adjusts responsively without breaking

---

Conclusion

The experiment successfully demonstrates how to create a responsive webpage using Bootstrap. The grid system ensures that content adjusts automatically across all screen sizes, while Bootstrap components improve appearance and usability.

Questions

1. What is the purpose of the Bootstrap Grid System?
2. What is the difference between `col-sm-4` and `col-md-4`?
3. Name any three Bootstrap components and their uses.
4. Why is Bootstrap considered mobile-first?
5. What class is used to create a responsive image?

Experiment No: 06	
Title: Write a JavaScript program to validate form inputs (e.g., email, empty fields).	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 6

Title: Write a JavaScript program to validate form inputs (e.g., email, empty fields).

Aim

To validate user input in an HTML form using JavaScript, ensuring that all required fields are properly filled and the email format is correct.

Objectives

- To understand the importance of client-side form validation.
- To learn how JavaScript can check for empty fields.
- To validate email format using JavaScript.
- To prevent form submission until all validations are successful.

Problem Statement

Create an HTML form with fields such as Name and Email. Write a JavaScript validation script that checks for empty fields and verifies correct email format. If validation fails, display appropriate error messages and stop form submission.

Theory

Client-Side Validation

Form validation performed in the browser before sending data to the server. It improves user experience and reduces server load.

JavaScript for Validation

JavaScript is used to:

- Check if required fields are empty
- Check email format using regular expressions
- Display alerts or error messages
- Stop form submission if invalid

Tags & Attributes Used

Tag / Attribute	Description
`<form>`	Defines form structure
`<input>`	Takes user input
`type="text"`	Accepts text input
`type="email"`	Accepts email input
`onsubmit=""`	Calls validation function during form submission
`<script>`	Contains JavaScript code
`document.getElementById()`	Accesses input fields
`alert()`	Displays message boxes
`return false`	Stops form submission if invalid

Common Validation Checks

1. **Empty Field Validation:**
   Ensure Name or Email fields are not left blank.
2. **Email Format Validation:**
   Uses JavaScript Regex such as:
   `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`

Test Cases

Test Case No.	Input	Expected Output
1	Name: “Riya”, Email: “riya@gmail.com”	Form submits successfully
2	Name left empty	Error message: “Name is required”
3	Email left empty	Error: “Email is required”
4	Invalid email: “abc.com”	Error: “Enter a valid email”
5	Both fields empty	Error messages for all fields

Conclusion

The experiment demonstrates how JavaScript can be used to validate form input on the client side. By checking for empty fields and incorrect email format, form validation ensures cleaner and more accurate data submission.

Questions

1. What is client-side validation?
2. How does JavaScript help in validating form fields?
3. Why do we use regular expressions for email validation?
4. What happens if `return false` is written inside the validation function?
5. What is the difference between `alert()` and displaying an on-page error message?

Experiment No: 07	
Title: Create a web page that uses JavaScript to display dynamic content using DOM	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 7

Title: Create a web page that uses JavaScript to display dynamic content using DOM.

Aim

To dynamically update and display content on a webpage using JavaScript and the Document Object Model (DOM).

Objectives

- To understand the concept of DOM (Document Object Model).
- To learn how JavaScript interacts with webpage elements.
- To modify HTML content dynamically using DOM methods.
- To display updated content without reloading the page.

Problem Statement

Create a webpage that contains a button. When the user clicks the button, JavaScript should change or display dynamic content (such as text, date, greeting message, or color changes) using DOM manipulation methods.

Theory

What is DOM?

The Document Object Model (DOM) represents the HTML document as a tree structure.
JavaScript can use this model to interact with and modify the webpage's elements.

Common DOM Methods Used

Method	Description
`document.getElementById()`	Selects an element by ID

Method	Description
`innerHTML`	Changes the HTML content of an element
`style.property`	Changes CSS styles dynamically
`createElement()`	Creates new HTML elements
`appendChild()`	Adds elements to the DOM
`addEventListener()`	Adds event handlers

Tags & JavaScript Concepts Used

Tag / Concept	Description
`<button>`	Triggers JavaScript function
`<p>` / `<div>`	Placeholder for dynamic text
`<script>`	Contains JavaScript code
`onclick`	Event to run JS function
DOM Tree	Structure that JS manipulates

Test Cases

Test Case No.	Input	Expected Output
1	Click “Show Message” button	Displays a message dynamically
2	Click again	Updates/changes the message
3	Display date/time	Shows current date/time without page refresh
4	Change style dynamically	Text color or size updates

Test Case No.	Input	Expected Output
5	Append new elements	Creates new DOM elements dynamically

Conclusion

The experiment demonstrates how JavaScript modifies webpage content dynamically using DOM manipulation. Without reloading the page, JavaScript can update text, change styles, create elements, and interact with users, enhancing webpage interactivity.

Questions

1. What is DOM in web development?
2. How does JavaScript access HTML elements?
3. What is the difference between `innerHTML` and `textContent`?
4. What is the purpose of `addEventListener()`?
5. Can DOM manipulation change CSS styles? Explain.

Experiment No: 08	
Title: Write a JavaScript program to validate form inputs (e.g., email, empty fields).	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 8

Title: Write a JavaScript program to validate form inputs (e.g., email, empty fields).

Aim

To validate user input in an HTML form using JavaScript, ensuring that all required fields are properly filled and the email format is correct.

Objectives

- To understand the importance of client-side form validation.
- To learn how JavaScript can check for empty fields.
- To validate email format using JavaScript.
- To prevent form submission until all validations are successful.

Problem Statement

Create an HTML form with fields such as Name and Email. Write a JavaScript validation script that checks for empty fields and verifies correct email format. If validation fails, display appropriate error messages and stop form submission.

Theory

Client-Side Validation

Form validation performed in the browser before sending data to the server. It improves user experience and reduces server load.

JavaScript for Validation

JavaScript is used to:

- Check if required fields are empty
- Check email format using regular expressions
- Display alerts or error messages
- Stop form submission if invalid

Tags & Attributes Used

Tag / Attribute	Description
`<form>`	Defines form structure
`<input>`	Takes user input
`type="text"`	Accepts text input
`type="email"`	Accepts email input
`onsubmit=""`	Calls validation function during form submission
`<script>`	Contains JavaScript code
`document.getElementById()`	Accesses input fields
`alert()`	Displays message boxes
`return false`	Stops form submission if invalid

Common Validation Checks

3. **Empty Field Validation:**
   Ensure Name or Email fields are not left blank.
4. **Email Format Validation:**
   Uses JavaScript Regex such as:
   `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`

Test Cases

Test Case No.	Input	Expected Output
1	Name: “Riya”, Email: “riya@gmail.com”	Form submits successfully
2	Name left empty	Error message: “Name is required”
3	Email left empty	Error: “Email is required”
4	Invalid email: “abc.com”	Error: “Enter a valid email”
5	Both fields empty	Error messages for all fields

Conclusion

The experiment demonstrates how JavaScript can be used to validate form input on the client side. By checking for empty fields and incorrect email format, form validation ensures cleaner and more accurate data submission.

Questions

1. What is client-side validation?
2. How does JavaScript help in validating form fields?
3. Why do we use regular expressions for email validation?
4. What happens if `return false` is written inside the validation function?
5. What is the difference between `alert()` and displaying an on-page error message?

Experiment No: 09	
Title: Design a PHP script to display "Welcome" message and current date & time	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 9

Title: Design a PHP script to display "Welcome" message and current date & time

---

Aim

To create a PHP script that displays a welcome message along with the current date and time.

Objectives

- To understand basic PHP syntax and server-side scripting.
- To learn how to display dynamic content using PHP.
- To use built-in PHP functions to fetch current date and time.
- To integrate PHP code into an HTML webpage.

Problem Statement

Design a PHP webpage that outputs a "Welcome" message to the user and displays the current date and time dynamically whenever the page is loaded.

Theory

PHP

PHP (Hypertext Preprocessor) is a server-side scripting language used to create dynamic web content.

Key PHP Concepts Used

Concept / Function	Description
<?php ... ?>	PHP code block inside HTML
echo	Outputs text or variables to the browser
date()	Returns current date and time in a specified format
" "	HTML line break to separate output lines

Common PHP Date Formats

Format	Example Output
d-m-Y	01-12-2025
l	Monday
h:i:s A	10:30:45 AM
Y-m-d H:i:s	2025-12-01 10:30:45

Test Cases

Test Case No.	Input	Expected Output
1	Load page	Displays “Welcome” message and current date & time
2	Refresh page	Updates date & time to current server time
3	Change date format	Date displayed in specified format
4	Remove PHP code	Page shows blank or HTML content only
5	Test on server	Page executes successfully on PHP-enabled server

---

Conclusion

The experiment demonstrates the use of PHP to create dynamic content. Using `echo` and `date()` functions, the webpage can display personalized messages along with real-time information such as the current date and time.

Questions

1. What is PHP and how is it different from HTML?
2. What is the purpose of `echo` in PHP?
3. How do you display the current date and time in PHP?
4. What is the significance of `<?php ... ?>` tags?
5. Can PHP code run on a client-side browser without a server? Why or why not?

Experiment No: 10	
Title: Write a PHP program to accept form input and display it using the POST method	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 10

Title: Write a PHP program to accept form input and display it using the POST method

---

Aim

To create a PHP program that accepts user input from an HTML form using the POST method and displays the submitted data on the webpage.

Objectives

- To understand HTML form submission using the POST method.
- To learn how PHP retrieves data from form fields.
- To display dynamic content submitted by the user.
- To understand the difference between GET and POST methods.

Problem Statement

Design an HTML form with fields like Name, Email, and Message. When the user submits the form using the POST method, the PHP script should capture the input data and display it on the same or another webpage.

Theory

HTML Form and POST Method

- `<form method="POST" action="filename.php">`
  The `POST` method sends form data in the HTTP request body, keeping data hidden from the URL.
- Input fields use `<input>` or `<textarea>` tags.
- Form submission triggers PHP to process the data.

PHP Variables and Superglobals

- `$_POST['field_name']` retrieves data sent via POST method.

- Example:
- `$name = $_POST['name'];`
- `echo "Name: " . $name;`

Tags & Functions Used

Tag / Function	Description
`<form>`	Defines the form structure
`<input type="text">`	Text input field
`<input type="email">`	Email input field
`<textarea>`	Multi-line text input
`<input type="submit">`	Submit button
`method="POST"`	Sends form data securely in HTTP request body
`$_POST['field']`	Accesses submitted field data
`echo`	Outputs text to browser

Test Cases

Test Case No.	Input	Expected Output
1	Name: “Riya”, Email: “riya@gmail.com”, Message: “Hello”	Displays all submitted values
2	Leave Name empty	Name field shows blank or validation message
3	Enter invalid email	Displays email as entered (no built-in validation unless JS applied)
4	Submit form multiple times	Each submission displays latest values
5	Verify POST method	Data is not visible in URL

Conclusion

The experiment demonstrates how PHP captures and displays user-submitted form data using the POST method. It emphasizes secure and structured data handling compared to the GET method and enables interaction between users and server-side scripts.

Questions

1. What is the difference between GET and POST methods?
2. How does PHP access data submitted via POST?
3. Why is POST preferred for sensitive information?
4. What happens if the form's `method` attribute is omitted?
5. How can you validate form data before processing in PHP?

Experiment No: 11	
Title: Implement a PHP program for string manipulation (reverse, length, substring)	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 11

Title: Implement a PHP program for string manipulation (reverse, length, substring)

Aim

To perform various string manipulation operations in PHP, including reversing a string, finding its length, and extracting a substring.

Objectives

- To understand PHP string handling functions.
- To perform basic string operations such as reverse, length, and substring extraction.
- To learn how to process textual data dynamically using PHP.
- To integrate PHP string functions into a webpage for interactive use.

Problem Statement

Create a PHP script that takes a string input and performs the following operations:

1. Reverse the string
2. Calculate and display the length of the string
3. Extract and display a substring from the original string

Display the results dynamically on the webpage.

Theory

PHP String Functions

PHP provides built-in functions for string operations:

Function	Description	Example
`strrev($string)`	Reverses a string	`"Hello"` → `"olleH"`
`strlen($string)`	Returns length of a string	`"Hello"` → `5`
`substr($string, start, length)`	Extracts substring	`"Hello"` from index 1, length 3 → `"ell"`

PHP Syntax

- PHP code is enclosed in `<?php ... ?>` tags.
- Use `echo` to display output on the webpage.
- Variables start with `$` symbol.

Tags & Functions Used

Tag / Function	Description
`<?php ... ?>`	PHP code block
`echo`	Displays output
`strrev()`	Reverses a string
`strlen()`	Returns string length
`substr()`	Extracts substring
` `	HTML line break for output formatting

Test Cases

Test Case No.	Input	Expected Output
1	Input: “Hello World”	Reverse: “dlroW olleH”, Length: 11, Substring (0-5): “Hello”
2	Input: empty string	Reverse: "", Length: 0, Substring: ""

Test Case No.	Input	Expected Output
3	Input: “PHP Programming”	Reverse: “gnimmargorP PHP”, Length: 15, Substring (4-10): “Program”
4	Input with numbers: “12345”	Reverse: “54321”, Length: 5, Substring (1-3): “234”
5	Input with special chars: “@OpenAI!”	Reverse: “!IAnepO@”, Length: 8, Substring (1-5): “OpenA”

Conclusion

The experiment demonstrates how PHP can manipulate strings dynamically using built-in functions. Reversing, calculating length, and extracting substrings are fundamental operations for text processing in web applications.

Questions

1. What is the purpose of `strrev()` in PHP?
2. How does `strlen()` function work?
3. Explain the parameters of `substr()` function.
4. Can PHP handle multi-byte or Unicode strings with these functions?
5. Give a practical example where string manipulation is useful in web development.

Experiment No: 12	
Title: Create a PHP script to store and display values in an array	
Date of performance:	**Date of Submission:**
Class: SE	**Roll No.:**
Signature:	

Experiment No 12

Title: Create a PHP script to store and display values in an array

---

Aim

To create a PHP program that stores multiple values in an array and displays them dynamically on a webpage.

Objectives

- To understand PHP arrays and their types.
- To store multiple values in an array.
- To access and display array elements using loops.
- To understand practical usage of arrays in web development.

Problem Statement

Design a PHP script that creates an array containing multiple values such as names, numbers, or colors. Display the elements of the array using a loop on the webpage.

Theory

PHP Arrays

An array in PHP is a variable that can hold multiple values. Arrays can be **indexed** or **associative**.

Types of Arrays

Type	Description	Example
Indexed Array	Elements are accessed using numeric indexes	`$colors = array("Red", "Blue", "Green");`

Type	Description	Example
Associative Array	Elements are accessed using keys	`$age = array("John"=>25, "Riya"=>22);`
Multidimensional Array	Array containing one or more arrays	`$matrix = array(array(1,2), array(3,4));`

PHP Functions & Concepts Used

Function / Concept	Description
`array()`	Creates an array
`echo`	Displays content on webpage
`for` / `foreach`	Loop through array elements
`count()`	Returns number of elements in array
` `	HTML line break for display

Example Concept

```
$fruits = array("Apple", "Banana", "Cherry");
foreach($fruits as $fruit){
    echo $fruit . "<br>";
}
```

Test Cases

Test Case No.	Input	Expected Output
1	Array: ["Red", "Green", "Blue"]	Displays Red, Green, Blue each on a new line
2	Empty array	No output displayed
3	Associative array: ["John"=>25, "Riya"=>22]	Displays John:25 and Riya:22

Test Case No.	Input	Expected Output
4	Multidimensional array: [[1,2],[3,4]]	Displays nested elements in row-wise order
5	Count array elements	`count($array)` returns correct number of elements

Conclusion

The experiment demonstrates how PHP arrays store multiple values efficiently and how to access them using loops. Arrays are fundamental data structures in PHP for handling lists, storing dynamic content, and managing structured data in web applications.

Questions

1. What is an array in PHP?
2. Differentiate between indexed and associative arrays.
3. How do you loop through an array in PHP?
4. How can you find the number of elements in an array?
5. Give a practical example where arrays are useful in web development.